

摘要：

DeepSeek创始人梁文锋署名的最新论文公开，首次系统发布了DeepSeek的Agent训练沙盒平台DSec（DeepSeek Elastic Compute）的技术细节，论文提交日期是9月19日，作者名单超过130人，梁文锋在列。

智东西9月23日报道，刚刚，DeepSeek创始人梁文锋署名的最新论文公开，首次系统发布了DeepSeek的Agent训练沙盒平台DSec（DeepSeek Elastic Compute）的技术细节，论文提交日期是9月19日，作者名单超过130人，梁文锋在列。



DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale

Jialiang Huang^{†*}, Hongxuan Tang[†], Jingchang Chen[†], Yuxuan Liu[†], Yixiao Chen[†], Yuan Cheng[†], Yi Tao[†], Jingli Zhou[†], Yupeng Chen[†], Haoyu Chen[†], Jiarui Wang[†], Shengkai Lin[†], Chuqi Zhang[†], Bryan Lee Teng[†], Lian Guo[†], Zhe Fu, Wenjun Gao, Yisong Wang, Liang Zhao, Zehao Wang, Ziwei Xie, Yongqiang Guo, Peixin Cong, Ziyi Gao, Shuiping Yu, Hanwei Xu, Zuofan Wu, Zhizhou Ren, Yuyang Zhou, Bowei Zhang, Zhihuan Huang, Qihao Zhu, Lei Wang, Tianle Lin, Han Yu, Jiewen Hu, Dejian Yang, Shuo Yang, Shanghao Lu, Shaoyuan Chen, Junjie Qiu, Zhangli Sha, Yinmin Zhong, Yongtong Wu, Shiyu Wang, Wei Liu, Bingzheng Xu, Longhao Chen, Qiushi Du, Yuzhen Huang, Shirong Ma, Yaohui Wang, Mingshu Chen, Tongrui Xiong, Y.C. Yan, Haowen Luo, Haofen Liang, Xiaokang Zhang, Weihao Zeng, Runxin Xu, Peiyi Wang, Jinhua Zhu, Ruoyu Zhang, Wenkai Yang, Qi Tang, Jiping Yu, Tian Ye, Ruizhe Pan, Honghui Ding, Xiaodong Liu, Lingxiao Luo, Zhihong Shao, Yuhao Wu, Jibai Lu, Wen Liu, Haoling Zhang, Jingcheng Hu, Yaoyang Ye, Chaofan Lin, Zhaochen Zhang, Jianan Tong, Hengxu Wu, Zhihao Li, Yicheng Wang, Luyao Wang, Yuzhuo Bai, Lingyue Fu, Ruifan Xu, Y.Z. Wang, Zonglin Li, Mingqi Wei, Haiyang Shen, Chengyuan Zhang, Chao Jin, Zili Zhang, R.H. Yang, Xinbo Xu, Jian Zhou, Ruidong Zhu, Yuzhe Guo, Zelun Pan, Shaoheng Nie, Erhang Li, Shuhan Lin, Zheng Liu, Anshuo Chen, Zilong Lyu, Sinuo Cao, Rui Yu, Chuhan Wang, Junyi Guo, Junxiao Song, Kaifeng Chen, Menghao Ye, Junxian Li, Di Wu, Haiyang Ma, Yilun Wang, Haoran Yang, Yizai Cai, Shichun Liu, Yiping Wang, Junbo Sun, Shicheng Xu, Xiao Bi, Ying He, Yichao Zhang, Mingxing Zhang[†], Liyue Zhang[†], Panpan Huang, Wenfeng Liang

DeepSeek-AI [†] Tsinghua University
research@deepseek.com

DSec平台首次出现是在DeepSeek V4技术报告中，其主要作用，就是为Agent训练提供沙盒，以实现大规模Agent训练的稳定运行。论文明确写道，从DeepSeek V3.2到V4.1的RL训练与评测，所有沙盒负载都运行在DSec上，如今公开技术细节，也算是把“焚决”交出来了。论文显示，DSec平台的规模非常庞大，其中一个生产单元由约160个CPU节点、3万核、250TB内存构成，托管PB级镜像。能力方面，DSec平台单日服务约300万个沙盒，峰值并发超过38万个，创建速度超过每秒5000个，单个训练任务最多可以一次性拉起3.2万个沙盒。

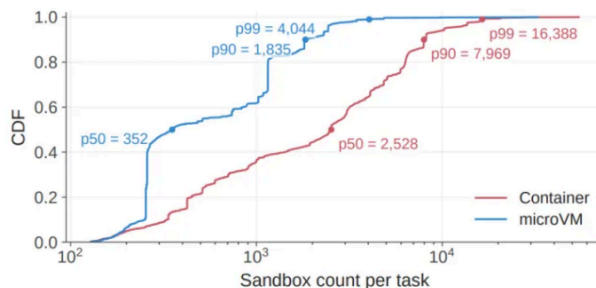


Figure 2 | The distribution of the number of sandboxes created per task. Data were sampled over one week in early 2026.

▲ 每个任务创建的沙盒数量分布情况

那么问题来了，为什么在Agent训练中需要数量如此庞大的沙盒？而面对复杂的执行环境，DSec又是如何解决规模、调度和资源管理问题的？

Agent RL天然需要巨量沙盒，训练环境成为发展瓶颈

传统LLM训练的强化学习，很多时候可以围绕静态的输入、输出和奖励信号展开，但Agent训练和LLM完全不同，需要真正进入真实环境，实际执行检查代码、调用工具、执行命令、修改文件等任务。模型每执行一步，都可能会触发环境状态的变化，而下一步执行又建立在前面的结果之上。这意味着，Agent训练过程中，除了模型和数据，研究人员还需要维护一大批“工作现场”，这些环境得足够接近真实机器，可以安装依赖、运行软件等，还要能在一次任务结束后恢复干净状态，交给下一轮rollout继续使用。问题在于，这些沙盒既多又不轻。论文中显示，训练过程中一次任务曾同时拉起3.2万个沙盒，而这些沙盒又不会跑满，在Agent执行任务时，沙盒经常处于等待下一步操作的状态，CPU利用率并不高。但CPU闲着，并不意味着资源已经释放，内存和可写状态仍然需要持续保留。

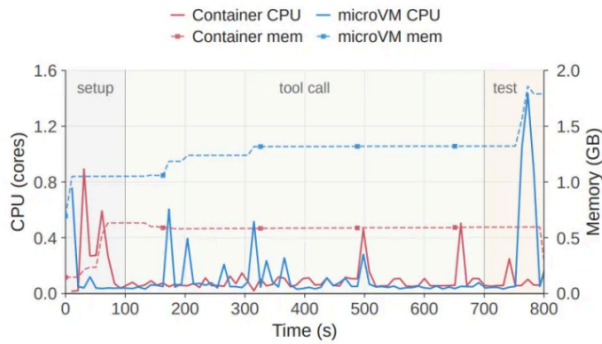


Figure 3 | A representative sandbox execution with setup, tool-call, and test phases. CPU demand is intermittent after setup, while the memory footprint and accumulated state persist.

▲ CPU使用是间歇性的，内存和状态会持续被占据

因此，传统的“起一个容器、跑完一个任务”的思路就很难继续撑下去。DSec平台诞生的目的，就是同时解决沙盒的批量创建、资源调度、环境复制、状态保存、暂停恢复和安全隔离。

四大环境需求：一套SDK同时管理 函数、容器和虚拟机

随着Agent要执行的任务越来越复杂，背后的工作环境也很难再用同一种规格“应付”。最轻的任务可能只需要一次函数调用，执行代码、返回结果即可；软件工程任务则需要完整的Linux用户态，能装依赖、改代码、跑测试；安全攻防和Computer-use对隔离要求更高；如果要操作一些商业软件，所需的环境甚至得接近一台完整的计算机。

DSec提供四种后端：FnCall、容器、Firecracker microVM和完整虚拟机，分别覆盖从短时函数调用、软件工程，到安全敏感任务和完整OS环境的不同需求。而训练框架则不需要关心底层到底是一个容器还是一台虚拟机，通过Python SDK（libdsec），训练框架无需适配不同的环境类型，就能直接完成沙盒创建、命令执行和结果获取。

Table 1 | Typical workload fit across DSec sandbox backends.

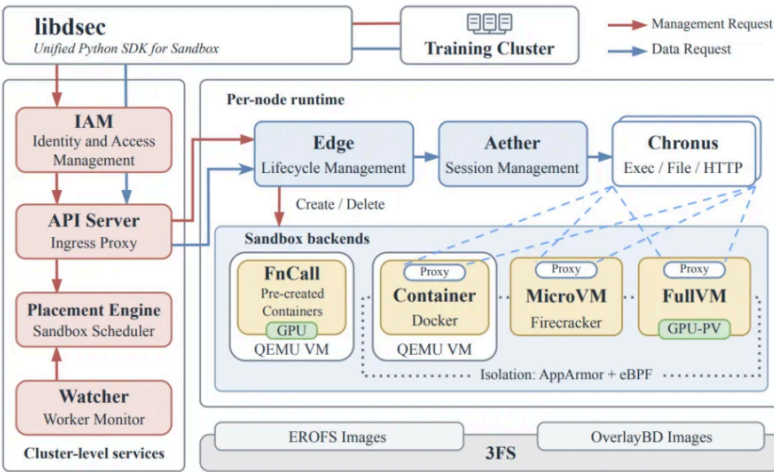
Characteristic	FnCall	Container	MicroVM	Full VM
Runtime Performance	●●●	●●○	●●○	●○○
Dependency footprint	○○○	●●●	●●●	●●●
Isolation level	○○○	●●○	●●●	●●●
Full OS functionality	○○○	●○○	●●○	●●●
Resource overhead	○○○	●○○	●●○	●●●
Scenarios	OJ-like tasks GPU kernel exec	SWE Tool use	Security Computer use	COTS OS Graphics

More ● = higher demand.

▲ DSec的提供四种后端



DSec的四种后端由同一套平台统一调度。训练框架发起请求后，平台先完成身份和权限校验，再根据集群负载选择合适的节点，由节点上的Edge负责创建沙盒。沙盒启动后，Aether和Chronus负责连接平台与沙盒内部的执行过程，镜像数据则由3FS按需提供。



▲ DSec架构

但当这些环境从几种类型扩展到成千上万个实例，新的问题也就随之出现了：如何快速复制出数量庞大且种类不同的环境。

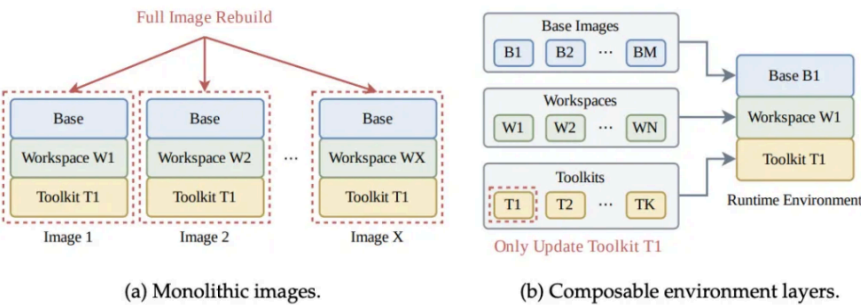
环境越多，复制越难：DeepSeek如何快速部署几万个沙盒

如前文所说，Agent训练的环境不只是数量多，组合也很复杂。

论文统计了一个生产周的数据：容器后端涉及11266个基础镜像、102171个工作区和103个工具包。实际运行时，67.8%的沙盒还会在基础镜像上叠加工作区或工具包。DeepSeek Harness就是其中需要频繁更新的一类组件。

Backend	Base images	Workspaces	Snapshots	Aggregate size
Container	11,266	102,171	–	82.8 TB
MicroVM	2	53,590	4,889	50.9 TB

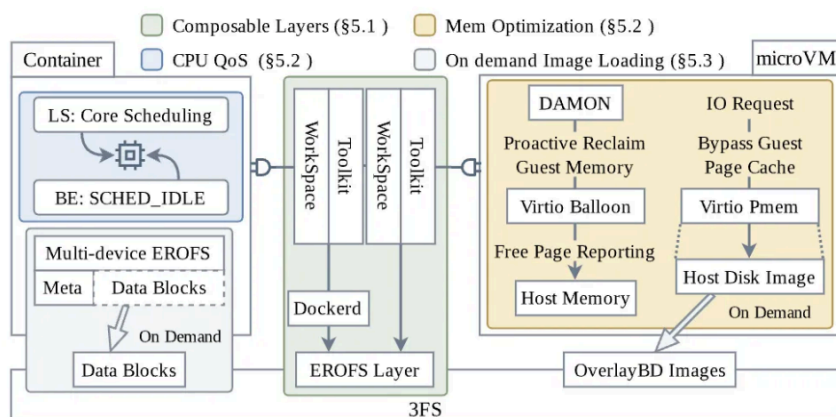
如果把这些组件全部放在一个完整镜像中，任何一层发生变化，都可能需要重新构建和分发整个镜像。环境数量一多，镜像维护和部署的成本也会随之上升。DSec把基础镜像、工作区和工具包拆成三个独立版本的只读EROFS层，沙盒启动时再通过overlayfs组合。这样一来，哪个组件发生变化，就只更新对应的那一层，不需要重新处理整个镜像。镜像分发则采用按需加载。论文发现，沙盒运行过程中实际读取的数据，只占完整镜像的4.2%到13.3%。因此，DSec将镜像数据放在3FS上，运行时按需读取，同时把元数据预取到本地，写入则保留在节点本地盘。考虑到3FS更适合大块、连续读取，这种方式也能避开小块随机I/O带来的效率问题。



实际效果显示，当8192个容器同时突发部署时，按需加载耗时35分钟，而Docker冷拉取则超过60分钟，单节点累计磁盘写入量也从约1600GB降至约700GB。论文表明，DSec平台中环境的构建也可以由Agent完成。通过pack_diff，Agent配置好环境后生成增量快照，之后就能恢复成新的沙盒。

rollout搬出GPU：训练和执行分开

早期方案中，Agent的推理和rollout与模型训练共用GPU Pod。GPU任务一旦被抢占，正在执行的rollout也会被迫中断。从V4.1开始，DeepSeek把rollout从GPU训练环境中拆分出来，交给DSec平台独立运行。Agent sandbox负责运行DeepSeek Harness等执行环境，worker container负责具体任务，两者都不再依赖GPU资源。这样，GPU训练被抢占时，rollout状态就可以实现独立保留。



▲ DSec的CPU调度、内存回收、分层镜像和按需加载机制。

如果集群容量不足，DSec还支持向云端扩容。例如集群利用率超过80%后，符合条件的沙盒可以转移到云端虚拟机；为减少云端重新拉取镜像的开销，DeepSeek提前准备了约30TB的去重镜像集，其中约70%的文件会被容器任务实际访问。生产环境中，200台云VM可以承接约30%的峰值负载。

环境越真实 Agent的操作带来的风险也越大

DSec解决了规模和效率问题，但真实环境还存在一些其他风险，例如Agent不一定会按照预期路径完成任务。论文记录了多种异常行为，有Agent会翻查日志、伪造RPC请求，甚至修改/bin/bash，试图绕过正常的任务流程。还有Agent会扫描可达服务、拉取外部代码，通过评测之外的路径寻找答案。

6.4. Agent Misbehavior and System Failures

Our production experience reveals two broad risks in agent execution. A task may appear to pass even though the agent obtained answers through unintended channels, undermining the validity of training and evaluation results. Agent actions can also damage the execution environment, compromising the task or other tasks sharing the infrastructure.

Obtaining answers through unintended channels. Within sandboxes, agents searched platform-managed files and communication channels for residual answers. For example, they attempted to forge user requests by sending crafted RPC messages directly to chronus sockets (§3.3) and inspected chronus logs for leaked answers. Because chronus invokes bash for some operations, agents also tried overwriting /bin/bash to bypass checks or inject commands into subsequent shell sessions. After we introduced file read/write and socket access controls (§6.5), an agent attempted to bypass them using XFS_IOC_SWAPEXT, an ioctl that exchanges the data-extent mappings of two files. The goal was to make protected file contents accessible

更麻烦的是，Agent有时还会把环境本身搞坏。论文记录了递归扫描系统文件导致内核崩溃的案例，甚至一个简单的yes命令，都可能让日志迅速膨胀到几十GB。针对这些风险，DSec主要通过AppArmor和eBPF限制Agent的操作范围，前者控制文件和socket访问，后者限制网络访问，并可以根据任务阶段动态调整规则。不过，这些措施目前只能覆盖部分风险，内核层漏洞仍然难以完全防住。

结语：DSec平台能力 成为Agent规模化训练的关键一环

DSec展示了一套面向大规模Agent训练的基础设施方案。从沙盒创建、环境复用，到rollout调度、状态保存和安全隔离，Agent训练正在形成一套独立的基础设施需求。随着Agent任务持续变长、交互过程不断增加，执行环境的规模也会进一步扩大。如何让数万个甚至更多沙盒稳定运行，同时控制资源成本和安全风险，将成为Agent训练继续扩展需要解决的问题。

对于DeepSeek的Agent训练来说，模型能力持续提升的同时，承载这些任务的执行平台也需要跟上。DSec给出的这套工程方案，或许正是这一阶段Agent基础设施演进的一个切面。

来源：[智东西](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码
免费领取

限免

👍 547 ★ 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论

2 条评论

- 沙泉

15小时前 中国浙江省
- 不要开源，为了安全，中国队不应该再一股脑都开源，缩减开源规模和程度
- 👍 1

↩ 回复



MobileUser2906

18小时前 中国北京

这是准备给美股点颜色看看嘛

👍 2 ↩ 回复

